

Especificación del Proyecto: Desarrollo de un Analizador Léxico para un Lenguaje Personalizado

1. Descripción General

El objetivo de este proyecto es diseñar un analizador léxico (lexer) para un lenguaje personalizado, que será utilizado en una aplicación creada por el estudiante. Este lexer transformará un archivo de texto que contenga un “programa” en una lista de tokens correspondientes a la gramática definida. La especificación del lenguaje será diseñada de manera informal utilizando ejemplos ilustrativos de código.

El proyecto incluye las siguientes fases:

- **Diseño de la especificación del lenguaje:** Definición de la sintaxis y semántica básica del lenguaje.
- **Implementación del analizador léxico:** Creación de un lexer funcional utilizando una herramienta de generación de analizadores (como ANTLR, Flex, o una implementación manual).
- **Pruebas y validación:** Verificación del correcto funcionamiento del lexer, incluyendo la identificación de errores léxicos.

2. Tareas del Proyecto

1. Especificación del Lenguaje Personalizado:

- Crear una descripción clara y concisa del lenguaje.
- Proveer ejemplos de código que ilustren la sintaxis de las instrucciones y patrones del lenguaje.
- Nombrar el lenguaje, asegurando que refleje su propósito o funcionalidad.

2. Especificación del Analizador Léxico:

- Diseñar un lexer que analice archivos de texto y extraiga tokens según la especificación del lenguaje.

- Incluir soporte para los siguientes elementos:
 - Identificadores (válidos e inválidos según las reglas del lenguaje).
 - Palabras reservadas.
 - Operadores.
 - Delimitadores (como `;`, `,`, `{`, `}`).
 - Números (enteros y/o flotantes según la especificación).

3. Implementación del Analizador Léxico:

- Desarrollar el lexer con las siguientes funcionalidades:
 - **Identificación de Tokens:** Convertir las entradas válidas en una lista de tokens.
 - **Manejo de Errores:** Detectar e informar errores relacionados con:
 - Identificadores no válidos.
 - Símbolos desconocidos.
 - **Mensajes de Error Informativos:** Cada error debe incluir:
 - Número de línea y columna donde se detectó el error.
 - Descripción clara del problema.

4. Pruebas del Lexer:

- Probar el lexer con varios ejemplos que incluyan:
 - Programas válidos que cumplan con la especificación del lenguaje.
 - Casos que generen errores, para verificar la utilidad de los mensajes de error.

3. Funcionalidades del Lexer

- **Reglas Léxicas:**

El lexer debe implementar las siguientes reglas básicas:

- Reconocer identificadores válidos (e.g., `variable1`, `my_function`).
- Detectar identificadores inválidos (e.g., `123abc`, `@var`).
- Diferenciar palabras reservadas (e.g., `if`, `else`, `while`) de identificadores.
- Reconocer símbolos válidos del lenguaje (e.g., operadores, delimitadores).
- Manejar números en formatos permitidos (enteros, decimales, etc.).

- **Manejo de Errores:**

- Detectar símbolos no reconocidos (e.g., \$, #).
- Proporcionar mensajes detallados con ubicación precisa del error.

- **Salida del Lexer:**

- Para cada entrada válida: Generar una lista ordenada de tokens en formato legible.
- Para entradas con errores: Mostrar los errores detectados y su localización.

4. Ejemplo de Salida del Lexer

Dada una entrada de programa:

```
if x > 10 then
  y = x * 2;
else
  z = y / 3;
```

El lexer debe generar:

Token List:

```
1: (KEYWORD, "if")
2: (IDENTIFIER, "x")
3: (OPERATOR, ">")
4: (NUMBER, "10")
5: (KEYWORD, "then")
6: (IDENTIFIER, "y")
7: (OPERATOR, "=")
8: (IDENTIFIER, "x")
9: (OPERATOR, "*")
10: (NUMBER, "2")
11: (DELIMITER, ";")
12: (KEYWORD, "else")
13: (IDENTIFIER, "z")
14: (OPERATOR, "=")
15: (IDENTIFIER, "y")
16: (OPERATOR, "/")
17: (NUMBER, "3")
```

18: (DELIMITER, ";")

En caso de errores:

Error List:

Line 1, Column 4: Unknown symbol '\$' detected.

Line 2, Column 1: Invalid identifier '123abc'.

5. Entregables del Proyecto

5.1 Archivos de Proyecto – Repositorio GitHub

El repositorio debe ser **público** y entregarse como enlace. Se sugiere la siguiente estructura:

analizador-lexico-[nombre-lenguaje]/

- |— README.md
- |— .gitignore
- |— LICENSE
- |— src/
 - |... |— main.[ext] ← punto de entrada / ejecutable principal
 - | |— lexer.[ext] ← lógica central del analizador léxico
 - |... |— tokens.[ext] ← definición de tipos de token
 - |... |— errors.[ext] ← manejo y reporte de errores léxicos
- |— grammar/
 - |... |— [nombre].g4 ← archivo de gramática (ANTLR, Flex, etc.)
 - | ← o reglas léxicas equivalentes según la herramienta elegida
- |— tests/
 - |... |— valid/
 - |... |... |— programa1.[ext-lenguaje] ← ejemplo de entrada válida
 - | | |— programa2.[ext-lenguaje]
 - |... |— invalid/
 - |... |— errores1.[ext-lenguaje] ← ejemplo con errores léxicos
 - | |— errores2.[ext-lenguaje]
- |— docs/
 - |... |— entregable_final.pdf
 - | |— diagrama_automata.png ← opcional pero recomendado
- |— capturas/

└─ captura01.png
└─ captura_n.png

[README.md](#) – Contenido mínimo requerido

El archivo `README.md` debe contener **al menos** las siguientes secciones:

[Nombre del Lenguaje] – Analizador Léxico

Breve descripción del lenguaje y propósito del lexer (1-2 líneas).

Información del Curso

- **Materia:** Compiladores (o nombre completo de la materia)
- **Institución:** [Nombre de la universidad]
- **Semestre:** 2026-1
- **Profesor:** [Nombre del profesor]

Integrantes del Equipo

Nombre	Matrícula
...	...

Descripción del Lenguaje

Resumen de la sintaxis y propósito del lenguaje diseñado.

Tokens Reconocidos

Lista de categorías de tokens soportadas (keywords, operadores, etc.)

Cómo ejecutar

Instrucciones paso a paso para compilar y correr el lexer.

Ejemplos de uso

Muestra de entrada y salida esperada.

5.2 Demostración Funcional – Documento PDF

El PDF debe entregarse dentro de `docs/entregable_final.pdf` en el repositorio.

Se requiere que incluya un **índice** y cubra las siguientes secciones:

Portada *(requerido)*

Nombre del proyecto, nombre del lenguaje diseñado, materia, institución, semestre, profesor e integrantes del equipo.

1. Introducción

Contexto y motivación del lenguaje creado; problema o dominio que busca resolver.

2. Objetivos

Objetivo general y objetivos específicos alineados a las fases del proyecto (especificación del lenguaje, implementación del lexer, pruebas y validación).

3. Especificación del Lenguaje

- Nombre del lenguaje y justificación.
- Descripción informal de la sintaxis con ejemplos de código ilustrativos.
- Tabla de tokens: categoría, descripción y patrón/expresión regular.

4. Implementación *(requerido)*

Código fuente comentado, organizado por módulo:

- 4.1 Definición de tokens y palabras reservadas
- 4.2 Reglas léxicas (expresiones regulares o gramática)
- 4.3 Lógica del analizador léxico
- 4.4 Manejo y reporte de errores (con número de línea y columna)

5. Demostración Funcional *(requerido)*

Capturas de pantalla anotadas que muestren:

- 5.1 Ejecución con un programa válido y su lista de tokens generada
- 5.2 Ejecución con entradas que contengan errores léxicos y los mensajes producidos
- 5.3 Al menos un caso con identificadores inválidos y/o símbolos desconocidos

6. Conclusiones

Aprendizajes individuales por integrante, dificultades encontradas durante el desarrollo y posibles mejoras o trabajo futuro.

7. Referencias

Bibliografía y fuentes externas consultadas (integridad académica).